Pitstop Oracle — ERD & Data Model

Version 3 | Separate the current file pipeline from future product persistence

1. Scope and assumptions

Separate the current file-oriented ML workflow from a possible future product database. The README describes raw files, a Parquet feature dataset, joblib models, and an optional future Postgres warehouse. It does not establish production accounts, user picks, leagues, or a live Postgres schema.

> Do not migrate to Postgres merely to complete an ERD. First validate the product loop and the need for durable multi-user state.

2. Current file-oriented workflow

Artifact	Role	Persistence note
Jolpica API	Historical and event data	Keep source IDs and fetch metadata
Open-Meteo	Weather inputs	Preserve forecast timestamp
data/raw	Raw ingested files	Useful for reproducibility
driver_race.parquet	Feature dataset	Version / fingerprint it
models/joblib	Model artifacts	Track model and training metadata
metrics report	Evaluation output	Store split and metric definitions

A database becomes useful when durable history, concurrent writes, user identity, or query requirements justify it.

3. Conceptual ML entities

Entity	Key attributes	Relationships
season	season_id, year	One season has many events
event	event_id, season_id, round, circuit_id, dates	Belongs to season
driver	driver_id, code, name	Appears in entries and results
constructor	constructor_id, name	Appears in entries and results
event_entry	event_id, driver_id, constructor_id	Event roster
session_result	event_id, session_type, driver_id, grid, position	Qualifying / race facts
weather_observation	event_id, observed_at, forecasted_for	Weather with time semantics
feature_snapshot	snapshot_id, event_id, stage, data_as_of	Input snapshot
feature_value	snapshot_id, driver_id, feature_name, value	Driver-level features
model_version	id, algorithm, params, artifact_uri	Immutable model identity
forecast_run	id, event_id, model_version_id, snapshot_id	One run per forecast
driver_forecast	forecast_run_id, driver_id, score, rank	Per-driver outputs
evaluation_run	id, split_definition, target	Evaluation metadata
evaluation_metric	evaluation_id, name, value, sample_size	Aggregate metrics

Use provider-independent canonical IDs internally. Keep provider IDs in mapping tables or source metadata.

4. Future product entities

Entity	Key attributes	Relationships
app_user	user_id, email, display_name	Owns submissions
prediction_submission	id, user_id, event_id, submitted_at, cutoff	One submission has picks
prediction_pick	submission_id, driver_id, predicted_rank	User's selected drivers
competition	id, name, season_id, rules_version	Contains members
competition_member	competition_id, user_id, joined_at	Many-to-many membership
submission_score	submission_id, rules_version, points	Score under versioned rules
notification_preference	user_id, channel, event_type, enabled	Opt-in notification settings

Keep user-generated predictions separate from model forecasts: they differ in authorship, timestamps, permissions, and retention.

5. Relationship sketch and constraints

Season 1—N Event; Event 1—N EventEntry; Driver 1—N EventEntry; Constructor 1—N EventEntry; Event 1—N FeatureSnapshot; FeatureSnapshot 1—N FeatureValue; ModelVersion 1—N ForecastRun; ForecastRun 1—N DriverForecast; AppUser 1—N PredictionSubmission; PredictionSubmission 1—N PredictionPick; Competition N—N AppUser through CompetitionMember.

• Unique event entry: (event_id, driver_id).

• Unique driver forecast: (forecast_run_id, driver_id).

• Unique competition membership: (competition_id, user_id).

• Validate pick rank range and one pick per driver per submission.

• Store timestamps in UTC and retain source-local timezone when relevant.

• Represent missing values as NULL / explicit unknown, not zero.

6. Indexes and provenance

• event(season_id, round) and event(event_date).

• session_result(event_id, session_type, position).

• feature_snapshot(event_id, forecast_stage, generated_at DESC).

• forecast_run(event_id, generated_at DESC).

• driver_forecast(forecast_run_id, rank).

• prediction_submission(user_id, submitted_at DESC) and (event_id, submitted_at).

• competition_member(user_id, competition_id) and reverse order.

For source facts, retain source name, provider ID, fetched_at, source timestamp, and optionally a payload hash. Version feature schemas, model artifacts, evaluation splits, and scoring rules.

7. Migration sequence and open questions

Stage	Work	Gate
A	Keep files, Parquet, joblib	Avoid infrastructure churn
B	Add reproducible manifests and metadata	Auditable pipeline
C	Persist forecast runs if history is needed	Forecast ledger use case
D	Introduce Postgres for canonical entities and product state	Real relational need
E	Add users, picks, scoring, leagues	Manual challenge validates demand

Verify whether IDs are canonical, forecasts persist today, feature snapshots are time-aware, and dataCollectionService contains migrations or a deployed schema. Required source audit: ingest.py, features.py, train.py, predict.py, run_pipeline.py, SQL migrations, and dataCollectionService.

The optional dataCollectionService is described in the README as a future Postgres warehouse. Do not treat it as deployed or authoritative without inspecting its source.